

CHAPTER - 1

Introduction

1.1 Mathematical Preliminaries

A. Sets and Subsets

A set is a well-defined collection of objects. For example: the set {red, yellow, blue} contains the names of primary colors. The individual objects are called member or elements of the set. We use the capital letters (A, B, C ...) for denoting the sets and smaller letters (a, b, c ...) to denote the elements of any sets. When 'a' is an element of the set 'A', we write $a \in A$, otherwise we write $a \notin A$. The set can be described in following three ways:

- *By listing its elements:* All the elements of a set are written by enclosing them within braces. For example: $A = \{3, 6, 9, 12, 15\}$
- *By describing the properties of the elements of the set:* The property of elements is called the predicate. For example: $B = \{x: x \text{ is a multiple of } 3\}$ or $B = \{x | x \text{ is a multiple of } 3\}$
- *By recursion:* The computational rule is defined for calculating each elements of the set. For example: the set of all natural numbers leaving a remainder 1 when divided by 3 can be described as: $\{a_n | a_0 = 1, a_{n+1} = a_n + 3\}$

Let A and B be sets. We say that B is a **subset** of A, and write $B \subseteq A$, if every element of B is an element of A. Two sets A and B are equal (we write $A = B$) if their members are the same. In practice, to prove that $A = B$, we prove $A \subseteq B$ and $B \subseteq A$. A set with no element is called an empty set, also called a null set or a void set, and is denoted by ϕ . We define the some **operations** on set as follows:

- *Union of A and B:* $A \cup B = \{x | x \in A \text{ or } x \in B\}$
- *Intersection of A and B:* $A \cap B = \{x | x \in A \text{ and } x \in B\}$
- *Complement of B in A:* $A - B = \{x | x \in A \text{ and } x \notin B\}$

B. Relations

The concept of relation is similar to the real life that compares one with the other. The relation made a pair of objects that is true in some cases and false in others; the statement 'x is less than y' is true if $x = 3$ and $y = 4$, for example, but false if $x = 3$ and $y = 2$.

A relation R in a set S is a collection of ordered pair of elements in S. When (x, y) is in R, we write xRy . When (x, y) is not in R, we write $xR'y$. For example: the relation R in set Z can be defined by xRy if $x > y$.

Properties of relations

The relations have generally three properties: reflexive, symmetric, and transitive. A relation R in a set S is called an equivalence relation if it is reflexive, symmetric, and transitive.

- A relation R in a set S is *reflexive* if xRx for every x in S.
- A relation R in a set S is *symmetric* if for x, y in S, yRx whenever xRy .
- A relation R in a set S is *transitive* if for x, y and z in S, xRz whenever xRy and yRz .

For example: Let us define a relation R in $\{1, 2, 3, \dots, 10\}$ by aRb if a divides b. R is reflexive and transitive but not symmetric ($3R6$ but $6R'3$).

- Reflexive relation: $1R1, 2R2, 3R3, 4R4, 5R5, 6R6, 7R7, 8R8, 9R9, \text{ and } 10R10$
- Transitive relation:
 - Let $x = 1, y = 2$, and $z = 4$ then $1R4$ whenever $1R2$ and $2R4$
 - Let $x = 2, y = 4$, and $z = 8$ then $2R8$ whenever $2R4$ and $4R8$
 - Let $x = 1, y = 3$, and $z = 6$ then $1R6$ whenever $1R3$ and $3R6$

C. Functions

Let X and Y are two sets. A *function* or *map* ' f ' from X to Y is a rule that assigns to each element x of X exactly unique element of Y , which is denoted by $f(x)$. The element $f(x)$ is called the image of x under ' f '. The function is denoted by $f: X \rightarrow Y$. The function can be defined in two ways:

- By giving the images (collection of values) of all elements X . For example: $f: \{1, 2, 3, 4\} \rightarrow \{a, b, c\}$ can be denoted by $f(1) = a, f(2) = c, f(3) = a, f(4) = b$.
- By computational rule which computes $f(x)$ once x is given. For example: $f: \mathbb{R} \rightarrow \mathbb{R}$ can be defined by $f(x) = x^2 + 2x + 1$ for every x in $\mathbb{R}$. ($\mathbb{R}$ denotes all the real numbers.)

Types of functions

- *One – to – one (or injective) function*: The function $f: X \rightarrow Y$ is said to be one-to-one if different elements in X have different images i.e. $f(x_1) \neq f(x_2)$ when $x_1 \neq x_2$. For example: $f: \mathbb{Z} \rightarrow \mathbb{Z}$ given by $f(n) = 2n$ is one-to-one but not onto.
- *Onto (subjective) function*: The function $f: X \rightarrow Y$ is said to be onto if every element y in Y is the image of some element x in X i.e. one-to-many relations.
- *One – to – one correspondence (bijection)*: The function $f: X \rightarrow Y$ is said to be one-to-one correspondence if f is both one-to-one and onto.

1.2 Alphabets and Languages

A. Alphabet

An alphabet is a non-empty set of *symbols* or *letters*, e.g. characters $\{a, b, \dots, z\}$ or digits $\{0, 1\}$ or ASCII characters etc. The alphabets are represented by the symbol Σ . Alphabets are important in the use of formal languages, automata and semi-automata. In most cases, for defining instances of automata, such as deterministic finite automata (DFAs), it is required to specify an alphabet from which the input strings for the automaton are built.

For example a common alphabet is $\{0, 1\}$, the *binary alphabet*. A finite string is a finite sequence of letters from an alphabet. For example, if we use the binary alphabet $\{0, 1\}$, the strings $(\epsilon, 0, 1, 00, 01, 10, 11, 000, \text{etc.})$ would all be in the Kleene closure (i.e. set of alphabets) of the alphabet (where ϵ represents the empty string). An infinite sequence of letters may be constructed from elements of an alphabet as well.

B. Strings

A string (or word) is a finite sequence of symbols chosen from some alphabet. For example: 01101 and 111 are strings from the binary alphabet $\Sigma = \{0, 1\}$.

- The string with zero occurrences of symbols is called *empty string* (denoted by ϵ).
- The number of positions for symbols in the string is called *length of a string*. The length of string is noted as: $w: |w|$. For example: $|011| = 3$ and $|\epsilon| = 0$

Generation of strings

If Σ is an alphabet, we can express the set of all strings of a certain length from that alphabet by using the exponential notation: Σ^k : the set of strings of length k , each of whose is in Σ .

Examples:

- $\Sigma^0: \{\epsilon\}$, regardless of what alphabet Σ is. That is ϵ is the only string of length 0

If $\Sigma = \{0, 1\}$, then:

- $\Sigma^1 = \{0, 1\}$

- $\Sigma^2 = \{00, 01, 10, 11\}$
- $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

Note: confusion between Σ and Σ^1

- Σ is an alphabet; its members 0 and 1 are symbols
- Σ^1 is a set of strings; its members are strings (each one of length 1)

Kleen star

Σ^* : The set of all strings over an alphabet Σ

- $\{0, 1\}^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$
- $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$

The symbol $*$ is called **Kleene star** and is named after the mathematician and logician Stephen Cole Kleene.

Note:

- $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \dots$
- Thus: $\Sigma^* = \Sigma^+ \cup \{\epsilon\}$

String concatenation

Strings can be concatenated yielding another string using the binary operation $(.)$, called *concatenation* on Σ^* . If $a_1a_2a_3 \dots a_n$ and $b_1b_2 \dots b_m$ are in Σ^* , then $a_1a_2a_3 \dots a_n . b_1b_2 \dots b_m = a_1a_2a_3 \dots a_nb_1b_2 \dots b_m$.

If x and y be strings then xy denotes the concatenation of x and y , that is, the string formed by making a copy of x and following it by a copy of y . For examples:

- $x = 01101$ and $y = 110$ then $xy = 01101110$ and $yx = 11001101$
- For any string w , the equations $\epsilon w = w\epsilon = w$ hold.

C. Language

Languages define a set of alphabets and associated meanings. A *formal language* L over an alphabet Σ is a subset of Σ^* , that is, a set of words over that alphabet. Sometimes the sets of words are grouped into expressions, whereas rules and constraints may be formulated for the creation of '*well-formed expressions*'. For example, the expression $0(0 + 1)^*1$ represents the set of all strings that begin with a 0 and end with a 1. A formal language is often defined by means of a formal grammar such as a regular grammar or context-free grammar, also called its formation rule.

If Σ is an alphabet, and $L \subseteq \Sigma^*$, then L is a (formal) **language** over Σ . A language (possibly infinite) set of strings all of which are chosen from some Σ^* .

Examples: The language may be Programming language C, English or French and others are:

- The language of all strings consisting of n 0s followed by n 1s ($n \geq 0$): $\{\epsilon, 01, 0011, 000111, \dots\}$
- The set of strings of 0s and 1s with an equal number of each: $\{\epsilon, 01, 10, 0011, 0101, 1001, \dots\}$
- Σ^* is a language for any alphabet Σ
- $\emptyset$, the empty language, is a language over any alphabet

- $\{\epsilon\}$, the language consisting of only the empty string, is also a language over any alphabet
 - Note: $\emptyset \neq \{\epsilon\}$ since $\emptyset$ has no strings and $\{\epsilon\}$ has one
- $\{w \mid w \text{ consists of an equal number of 0 and 1}\}$
- $\{0^n 1^n \mid n \geq 1\}$
- $\{0^i 1^j \mid 0 \leq i \leq j\}$

Important operators on languages

- The **union** of two languages L and M, denoted $L \cup M$, is the set of strings that are in either L, or M, or both. For example: If $L = \{001, 10, 111\}$ and $M = \{\epsilon, 001\}$ then $L \cup M = \{\epsilon, 001, 10, 111\}$
- The **concatenation** of languages L and M, denoted $L.M$ or just LM , is the set of strings that can be formed by taking any string in L and concatenating it with any string in M. For example: If $L = \{001, 10, 111\}$ and $M = \{\epsilon, 001\}$ then $L.M = \{001, 10, 111, 001001, 10001, 111001\}$
- The **closure** of a language L is denoted L^* and represents the set of those strings that can be formed by taking any number of strings from L, possibly with repetitions (*i.e.*, the same string may be selected more than once) and concatenating all of them. For examples:
 - If $L = \{0, 1\}$ then L^* is all strings of 0 and 1
 - If $L = \{0, 11\}$ then L^* consists of strings of 0 and 1 such that the 1 come in pairs, *e.g.*, 011, 11110 and ϵ . But not 01011 or 101.